Design and Implementation of Automation Deployment Tool for Power Grid Dispatching and Control System

Jun Hu[1,2] , Bing Zhang[1,2] , Jixiang Lu[1,2] , Liyan Xu [1,2] , Meifang Hou [1,2]
1. NARI Group Corporation(State Grid Electric Power Research Institute), Nanjing 211106, China;
2. NARI Technology Co. Ltd., Nanjing 211106, China
hujun@sgepri.sgcc.com.cn, zhangbing3@sgepri.sgcc.com.cn, lujixiang@sgepri.sgcc.com.cn, xuliyan@sgepri.sgcc.com.cn, houmeifang @sgepri.sgcc.com.cn

***Abstract*—To improve the quality and efficiency of software deployment of power grid dispatching and control system, an automation deployment tool is designed. In the overall system architecture, a JSON-RPC-based access service layer is designed to achieve high reuse of the back-end service under C/S and B/S architectures. This paper analyzes the structural characteristics of each module, and focuses on the implementation of JSON-RPC service and GUI under the C/S architecture. Finally, the function of the automation deployment tool is verified by testing.**

Keywords—Automation Deployment; Power Grid Dispatching and Control System; JSON-RPC

I. INTRODUCTION

Smart grid dispatching and control system is a highly integrated and intensive system for dispatching production business, which provides technical support for monitoring, analysis, control, planning, operation evaluation and dispatching management business of grid operation [1]. In order to support the dispatching operation of strong interconnected large-scale power grid, the overall framework with "physical distribution and logical integration" is proposed, which is composed of analysis and decision-making center, model cloud, and distributed monitoring subsystems [2]. The centralized deployment of the analysis and decision-making center and model cloud has made operation and maintenance difficult. It is therefore necessary to innovate the system operation and maintenance technology, improve the operation and maintenance management efficiency, and ensure the stable and reliable operation of the control system [2].

At present, the software delivery of power grid dispatching and control system is mainly manual deployment，which is time-consuming and inefficient. When the number of servers grows, the consistency of operation is not guaranteed and the probability of error is high, which renders it difficult to meet the operation and maintenance requirements of the above-mentioned large-scale power grid dispatching and control system (referred to as the new generation system). In the IT field, it has become a common practice for the operation and maintenance of large-scale distributed systems to improve the efficiency and quality of software deployment by implementing release engineering and establishing an automatic deployment system to ensure fast and reliable delivery of software [3-5]. Therefore, it is very important and urgent to design and implement an automation deployment tool for the new generation system, which can replace the manual deployment.

II. REQUIREMENT ANALYSIS

A. functional requirement

Software deployment includes installation, configuration, enabling, deactivation, update, reconfiguration, redeployment and deletion [6]. Automation Deployment refers to the deployment of software to the running environment with automated tools, including test environment, staging environment and production environment [7]. Software automation deployment generally involves configuration file, scripts and batch operation [8].

The software deployment of the power grid dispatching and control system is usually divided into two steps, the first being to copy the program to the corresponding node, and the second being to correctly configure the relevant configuration files. The traditional power grid dispatching and control system generally adopts the master and standby architecture, and its software upgrading process includes:

1. Randomly select a standby machine to perform software update and restart to be in the standby state.

2. If it passes the test, it switches to the master for retest, otherwise it rolls back.

3. If it passes the test as the master, continue upgrading the other servers. If it fails, it switches from the master to the standby and rolls back.

In the new generation system, the analysis and decision-making center and model cloud are centrally deployed, and their application provides services in the form of cloud. Therefore, the software upgrade needs to meet the requirements of zero downtime deployment [10-11], which means that the software upgrades without stopping its services.

978-1-7281-4390-3/20/$31.00 ©2020 IEEE

Based on the above analysis, the functional requirements of the automation deployment tool mainly include:

1. Configuration files management. The deployment tool supports defining the configuration file template, and then dynamically generating the configuration file using the template engine [9], instead of manual modification, to reduce human errors.

2. Automation deployment. The deployment tool supports the definition of series of operations such as deletion, installation, configuration, service start / stop, etc. for batch execution. The deployment task is automatically scheduled and executed in the backend, and logs are recorded. When the deployment of the new version fails, it can be quickly rolled back. A graphical human-machine interface is provided.

B. Non- functional requirements

According to the safety zoning requirements of the regulations on the safety protection of electric power secondary system, the subsystem with real-time monitoring function in the power grid dispatching and control system is located in the safety zone I - control zone, the dispatching plan subsystem is located in the safety zone II - non control zone, and the dispatching management subsystem is located in the safety zone III - management zone; HTTP and other general network services are prohibited in the control zone [12]. Therefore, to ensure security, in zone I and II, the deployment tool itself does not use general services such as HTTP, and does not install agents on the managed servers. In addition, the deployment tool must be scalable and upgradable, which itself is simple and supports container deployment.

III. SYSTEM DESIGN

A. Overall objectives

According to the above requirement analysis, the overall design goal is to implement an automation deployment tool of power grid dispatching and control system, which can operate independently, expand its functions, has a graphical human-machine interface, defines deployment projects and tasks, is suitable for C/S and B/S scenarios, and meets the safety protection requirements.

B. systeml design

As shown in Figure 1, the automation deployment tool software is divided into three layers: front-end interface, access service and back-end service. The front-end interface includes C/S graphic terminal and B/S browser. The access service layer is also divided into C/S and B/S, respectively referred to as C/S service and B/S service in this paper. And the back-end service mainly includes deployment service, monitoring service and relational database.

In order to meet the needs of C/S and B/S application scenarios, and realize as much reuse as possible, an access service layer is abstracted. The access service layer adapts the front-end C/S graphic terminal via C/S services and the front-end B/S browser via B/S services. The access service layer shields the front-end differences, so the back-end services are not affected at all. C/S service is based on TCP protocol and B/S service is based on HTTP protocol, but the upper layer uses JSON-RPC protocol to implement remote procedure call. Combined with the modular design, the specific functions of the deployment service can be implemented once, which can be shared between the C/S and B/S.

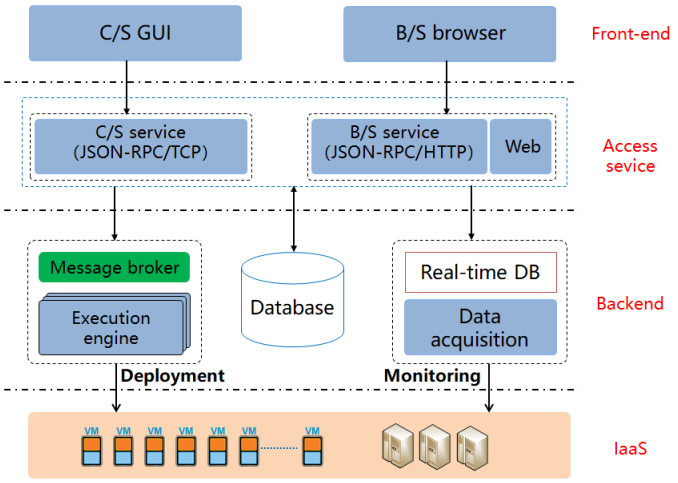

Fig. 1. Architecture Diagram

The access service layer realizes asynchronous calling and distributed task management of deployment service using message queue, ensuring that the same task is executed only once. The execution engine of the deployment service can deploy multiple parallel processing nodes as required, so the processing capacity can be scalable. Access service layer and monitoring service share data using real-time database. The C/S graphic terminal uses QT technology to realize cross platform deployment.

IV. IMPLEMENTATION

Python is an object-oriented, dynamic programming language. It is simple, flexible, portable, efficient and suitable for script processing, and is widely used in the field of operation and maintenance. Python has rich extension modules and third-party support packages. PyQt [13] is a GUI library, with which you can quickly develop a graphical interface to meet the needs of users. Tornado [14] is a service framework, with which you can quickly develop TCP based service programs and HTTP based WEB applications. Celery [15] is a distributed task management framework, with which you can quickly achieve distributed task management. Therefore, the python language is chosen to develop the automation deployment tool of this paper.

A. Deployment Services

The deployment service consists of an execution engine and distributed task management.

1) Execution engine

The execution engine is based on Ansible[16], which is an open source automation tool. It extends the function of generating host list through CMDB (Configuration Management Database). Based on Ansible's roles mechanism,

it develops an extension module for the installation of power grid dispatching and control system, which can be directly referenced when writing deployment scripts. As shown in Figure 2, the execution engine transforms the deployment script into executable operations, distributes it to the host defined in the host list for execution through SSH protocol, and automatically deletes it after execution.

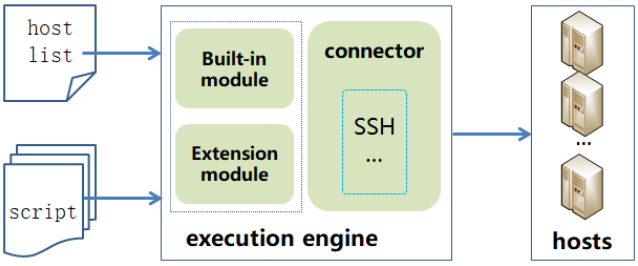

Fig. 2. Execution Engine

2) Distributed task management

Asynchronous message transmission using message queue is a common design pattern to realize application decoupling in distributed system. OpenStack cloud platform uses message queuing to coordinate the operation and status information between services [17-18]. Celery is a task queue, which focuses on real-time processing and supports task scheduling. Distributed task management is implemented based on the Celery, whose architecture is shown in Figure 2. It supports asynchronous tasks and scheduled tasks. Asynchronous tasks are usually triggered and submitted by users, and scheduled tasks are automatically triggered according to the predefined schedule. The task execution unit, as a consumer, listens to the message queue, reads the task message and executes it. The execution result is saved in the database so that it can be traced.

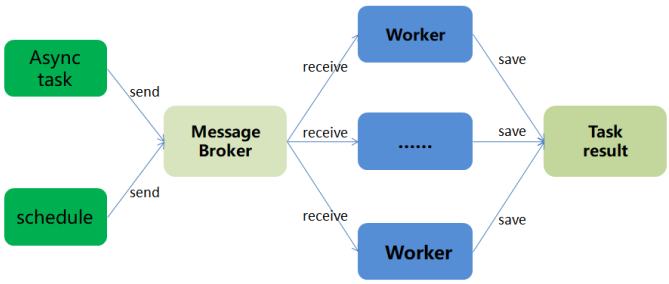

Fig. 3. Distributed Task Management

B. Access service

JSON-RPC is a stateless and lightweight remote procedure call protocol [19]. It uses JSON to transmit data, and the underlying protocol can be based on HTTP, TCP, etc. The former uses HTTP, with Spring Boot and other Web frameworks. It is convenient to implement the functions of data transmission and service invocation. The latter is directly based on TCP, which needs to deal with JSON data transmission, function registration and callback. This paper mainly expounds the implementation of the latter.

1) Custom data transfer protocol

The custom protocol format is shown in Figure 4. A complete data includes start flag, type, length, session ID, data and end flag. The start flag, type, length, session ID and end flag are of fixed length, which are used to ensure the integrity and security of the data package.

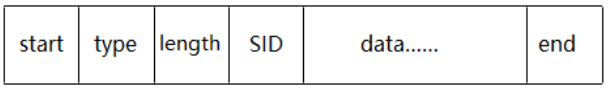

Fig. 4. self-defined protocol

2) JSON-RPC service based on TCP

JSON-RPC service is based on the third framework tornado implementation of Python. As shown in Figure 5, Tornado's TCPServer class encapsulates the basic socket functions. The JSON-RPC service class, JsonRpcServer, inherits from TCPServer and overrides methods such as connection and stream processing. Tornado's SSLIOStream class encapsulates the support for SSL (secure socket layer), which is convenient for secure communication. The JsonRequest class implements the functions of callback function registration, call back and response data generation. The design and implementation make full use of tornado's asynchronous IO and coroutine features, making the structure simple and reliable.

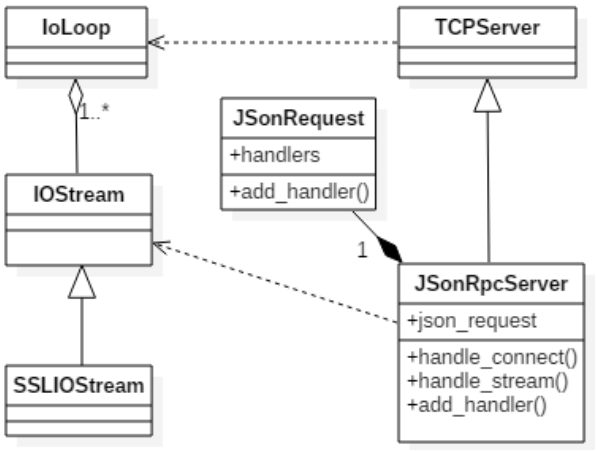

Fig. 5. JSON-RPC service class diagram

3) JSON-RPC client based on TCP

The implementation of JSON-RPC client makes full use of the dynamic characteristics of Python language. The Client class inherits from the object class, whose functions such as __get__ and __call__ are overridden so that it can intercept the call. When calling a method that does not exist, the client converts the method name and parameters to JSON-RPC packets, then send them to the server and wait for the return result. This implementation is completely transparent to the caller and easy to use.

C. user interface

The web interface is developed in MVVM(model view view model) mode with vue.js as the front-end framework. The front-end and back-end are separated, and the back-end service is called through the remote interface. The architecture is simple and reliable, and the debugging is convenient.

The C/S graphic terminal is based on PyQt5 so that it can be deployed across platforms. Its QThread class can send signals between the sub thread and the main user interface thread. Remote procedure call as an independent task is placed in the sub thread, which can avoid blocking the main thread due to time-consuming operation. As shown in Figure 6, the user triggers the call, starts the task thread and registers the signal. When the call is finished, a signal is sent to update the user interface in the main thread.

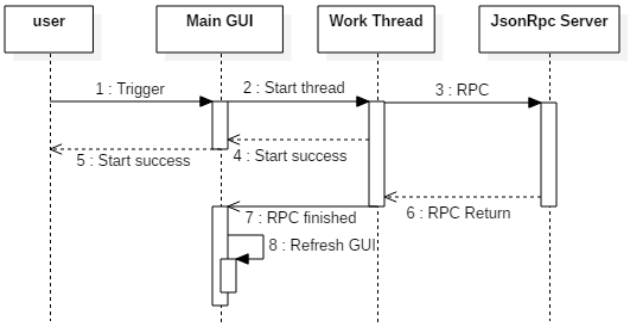

Fig. 6. Remote Procedure Call Sequence Diagram

V. TESTING SITUATION

At present, the basic function of deployment tool prototype has been developed and tested in the demo environment of the new generation system.

A. testing environment

The demo environment is built on the IaaS cloud. As shown in Figure 7, it includes 4 monitoring systems, 2 Analysis and decision-making systems and 1 real-time data platform. The IaaS cloud creates an isolated internal network for each system, and three external networks: human-machine, resource synchronization and data collection. Each system is equipped with at least one human-machine server. These Servers are interconnected via human-machine network. The IaaS cloud creates a virtual machine as a deployment server, which is connected to the human-machine network. Using SSH protocol, it can access all virtual machines of seven systems in the demo environment. For simplicity, only the human-machine network used for deployment is shown in the figure.

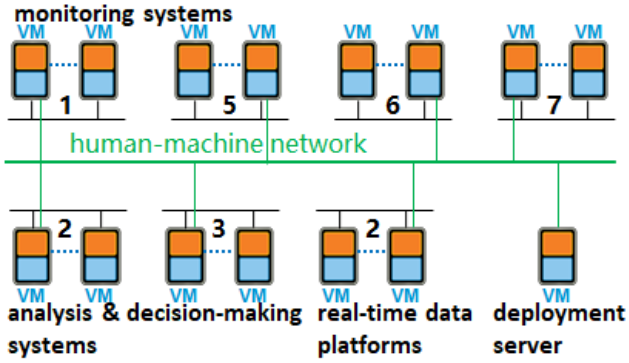

Fig. 7. testing environment

B. Test scenario

Test scenario simulates support platform software upgrade, which demonstrates the software deployment process from test environment to staging environment, then to production environment.

1. Taking monitoring system 1 as test environment to upgrade the software from v1.01 to v1.02;

2. Rolling back to v1.01 when finding bugs in v1.02;

3. Fixing the bugs to form v1.03, and to deploy v1.03 in monitoring system 1;

4. Taking analysis system 2 as the staging environment to deploy v1.03;

5. Upgrading the other systems as production environment to v1.03 one by one.

C. Testing process

Preparation should be made before testing, which includes: uploading software package; Define a project for each system, including upgrade steps, deployment scripts and host list files. The number of servers, host names, etc. of each system in the demo environment are different, and the configuration is also different, but the deployment process is the same. The upgrade steps are shown in Table 1 and the parameters of the host list file are shown in Table 2. During deployment, the software version is controlled by modifying the value of the software version variable ver.

TABLE I. UPGRADE STEPS

No.	Step	Host list	Script
1	stop	/p01/hosts	/p01/stop.yml
2	update	/p01/hosts	/p01/update.yml
3	start	/p01/hosts	/p01/start.yml

TABLE II. DEPLOYMENT PARAMETERS

No.	Parameter	Value
1	prod_name	nusp
2	ver	1.01
3	home_dir	/home/nusp

This demo environment consists of 7 systems, with 35 virtual machines in total. It usually takes about 1 to 2 working days to deploy manually. Using the automation deployment tool, the deployment script is written and verified in advance, and the deployment task is submitted with one click during execution, and the execution engine copies the software to each server simultaneously. Several experiments have shown that the execution time of such small-scale deployments is generally less than 1 hour, and the deployment process can be repeated and traced. Compared with manual deployment, automation deployment has obvious advantages in efficiency and operation consistency. Only one scenario is tested this time, and more scenarios will be tested with the development of new generation system.

VI. CONCLUSIONS

The new generation system architecture is complex, so manual deployment is inefficient, error prone, and difficult to meet its operation and maintenance requirements. The test results show that the automation deployment tool designed in this paper implements one click deployment, which makes software deployment and upgrade simple, fast, reliable and traceable, and can meet the deployment requirements of different scenarios such as C/S and B/S, with practicability and generality. The automation deployment tool, which can effectively solve the operation and maintenance problems related to the software deployment and upgrade of power grid dispatching and control system, has a wide application prospect.

ACKNOWLEDGMENT

This work is supported by Science and Technology Project of State Grid Corporation of China (Research and Application of Key Technologies for Test Verification and Integrated Operation and Maintenance of Dispatching Automation System).

REFERENCES

[1] XIN Yaozhong, SHI Junjie, ZHOU Jingyang, et al. Technology Development Trends of Smart Grid Dispatching and Control Systems[J].Automation of Electric Power Systems,2015,39(1):2-8. DOI: 10.7500/AEPS20141008024.

[2] XU Hongqiang, YAO Jianguo, YU Yijun . Architecture and Key Technologies of Dispatch and Control System Supporting Integrated Bulk Power Grids [J] Automation of Electric Power Systems,2018,42(6):1-8. DOI: 10.7500/AEPS20170120004. DOI:10.7500/AEPS20170617001.

[3] Adams, Bram & Bellomo, Stephany & Bird, Christian & Marshall-Keim, Tamara & Khomh, Foutse & Moir, Kim. (2015). The Practice and Future of Release Engineering: A Roundtable with Three Release Engineers. Software, IEEE. 32. 42-49. 10.1109/MS.2015.52.

[4] Shahin, Mojtaba & Ali Babar, Muhammad & Zhu, Liming. (2017). Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices. IEEE Access. PP. 10.1109/ACCESS.2017.2685629.

[5] Besty Beuer, Chris Jones. Site Reliability Engineering: How Google Runs Production Systems [M]. O'Reilly Media, 2016.

[6] Alan Dearle. Software deployment, past, present and future [C]. International Conference on Software Engineering (Future of Software Engineering) , 2007.

[7] Humble J , Farley D , Farley D . Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation[M]. Addison-Wesley, 2010.

[8] Vanish Talwar. Approaches to Service Deployment [J]. IEEE Internet Computing, 2005, 9 (2) :70 – 80

[9] template engine [OL]. https://baike.baidu.com/item/%E6%A8%A1%E6%9D%BF%E5%BC%95%E6%93%8E

[10] Deploying Applications to AWS Elastic Beanstalk Environments [OL]. https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.deploy-existing-version.html

[11] Michael T.Nygard. Release It!: Design and Deploy Production-Ready Sofware [M]. POSTS & TEKECOM PRESS, 2015

[12] XIN Yaozhong, TAO Hongzhu, SHANG Xuewei, et al. Support platform of Smart Grid Dispatching and Control System [M]. Beijing,CHINA ELECTRIC POWER PRESS 2016.

[13] PyQt [OL]. https://wiki.python.org/moin/PyQt

[14] The Tornado Authors. Introduction [OL]. http://www.tornadoweb.org/en/stable/guide/intro.html

[15] Celery - Distributed Task Queue [OL]. http://docs.celeryproject.org/en/master/index.html

[16] WHY ANSIBLE?[OL].https://www.ansible.com/overview/it-automation

[17] WANG Yuan, LIU Chuan-chang. Design and Implementaiton of Business Automatic for Cloud Computing [J]. Computer Engineering & Software, 2014, 35(9):66-72

[18] OpenStack. Message queue [OL] https://docs.openstack.org/mitaka/install-guide-ubuntu/environment-messaging.html. 2019-03-25

[19] JSON-RPC Working Group. JSON-RPC 2.0 Specification [OL]. https://www.jsonrpc.org/specification. 2013-01-04